AMERICAN UNIVERSITY OF BEIRUT

MAROUN SEMAAN FACULTY OF ENGINEERING & ARCHITECTURE

LAB 010

CI/CD WITH GITHUB ACTIONS

Part 0 — Read Before You Start

This lab is long. It is long because CI/CD touches many moving parts: Git, YAML, Docker, IAM, Kubernetes, AWS, security scanning, and Terraform. Rather than skip explanations, this edition expands every step — what the command does, why it is written that way, what happens if you get it wrong, and how to tell whether it worked.

Read Part 0 fully before running any commands. It sets up the mental model you need for the rest of the lab.

0.1 — What You Are Building

By the end of this lab, the workflow on your project will look like this:

1. You edit code on your laptop.
2. You commit and push to a branch on GitHub.
3. GitHub detects the push and starts a pipeline — a sequence of automated jobs.
4. The pipeline runs tests. If any test fails, the pipeline stops.
5. The pipeline builds Docker images for your frontend and backend.
6. The pipeline scans those images for security vulnerabilities. If a critical CVE is found, the pipeline stops.
7. The pipeline pushes the images to Amazon ECR.
8. The pipeline tells Kubernetes to update the running application to use the new images.
9. If you pushed to dev, the deployment happens automatically. If you pushed to main, the pipeline pauses and waits for a human to approve the production deploy.

You never run docker build, docker push, or kubectl apply by hand again. The same sequence runs the same way, every time, for every developer on the team.

0.2 — Your Starting State and Prerequisites

This lab continues from Lab 009 (Kubernetes on EKS). Lab 009 ended by asking you to delete the cluster and ECR repositories to avoid charges. That was correct. This lab assumes you are starting with no cluster, no ECR repos, and no running application, and guides you through recreating them.

You should already be comfortable with the following. If any feel shaky, review Lab 009 before continuing — this lab provides quick-reference commands but will not re-explain the concepts:

- Creating an EKS cluster with eksctl and understanding what it provisions (VPC, subnets, control plane, node group)
- Creating ECR repositories, logging in with the AWS CLI, and pushing Docker images
- Writing Kubernetes manifests: Deployment, Service, and how ClusterIP vs LoadBalancer differ

- Basic kubectl usage: get, describe, logs, apply, delete, exec
- Basic Git: commit, push, branch, checkout, merge

0.3 — Tools You Need Installed Locally

Tool	Version and Purpose
AWS CLI	v2 or later. Runs aws eks, aws ecr, aws sts commands. Authenticated via aws configure.
kubectl	v1.28+ is fine. Talks to the Kubernetes API.
eksctl	v0.160+ recommended. Creates and deletes EKS clusters.
Docker	Any recent version. Builds and pushes images.
Git	Any recent version. Pushes code to GitHub.
Node.js	v18+. Used only for running tests locally (optional).
Terraform	v1.6+ for Part 11. Required only if you do the Terraform section.

0.4 - 0.5 — Cost Awareness

This lab uses paid AWS resources. Here is what each resource costs at the time of writing:

Resource	Approximate Cost
EKS control plane	$0.10/hour — about $2.40/day, $72/month if left running
EC2 worker nodes (t3.medium × 2)	About $0.08/hour total — about $1.92/day
ECR storage	$0.10/GB/month — negligible for this lab
NAT Gateway (if created by eksctl)	$0.045/hour plus data transfer — can be significant
Network Load Balancer	About $0.025/hour per hour of use
S3 and DynamoDB (Terraform state)	Pennies for the volume in this lab

> ⚠️ **Warning**
> A full lab cluster left running for one week costs roughly

Resource	Approximate Cost
$30. A cluster forgotten for a month is about $120. Delete your cluster as soon as you finish each work session, even if you plan to return later. You can recreate it in 20 minutes; you cannot get a refund on idle charges. Set a phone reminder right now to check your AWS console every evening until you finish the lab.	

0.6 — Structure of This Lab

The lab has 14 parts, grouped in three phases:

10. **Phase A — Foundations** (Parts 1–5). Concepts, infrastructure, application code, Kubernetes manifests, and GitHub setup. No pipeline yet.
11. **Phase B — Build the Pipeline** (Parts 6–9). Write workflow files incrementally: unit tests, integration tests, build/scan/push, deploy to dev, deploy to prod.
12. **Phase C — Production Readiness** (Parts 10–14). Rollback, Terraform pipelines, branch protection, submission, and cleanup.

Each part builds on the previous one. Do not skip ahead. If something goes wrong in Part 8, the fix is often in a step you missed earlier.

Part 1 — CI/CD Concepts

1.1 — The Problem Manual Deployment Causes

Before talking about what CI/CD is, understand what it replaces. In previous labs, deploying a code change meant running a sequence of manual commands:

13. Edit the code locally.
14. Run docker build to produce a new image.
15. Tag the image for ECR.
16. Log in to ECR with aws ecr get-login-password.
17. Run docker push to upload the image.
18. Update the Kubernetes manifest with the new image tag.
19. Run kubectl apply or kubectl set image.
20. Watch kubectl get pods until everything is healthy.
21. Test the app in a browser.

This is about fifteen minutes of work even when nothing goes wrong. It goes wrong often. Let's look at common failure modes:

Failure mode 1: Forgetting a step

You edit code, build a new image, but forget the docker push. You run kubectl set image with the new tag. Kubernetes tries to pull an image that doesn't exist in ECR. Pods enter ImagePullBackOff. The app is now broken for users until you realize what happened.

Failure mode 2: Human inconsistency

You deploy from your laptop with Node 20 installed. Your teammate deploys from their laptop with Node 18. The build output is different. One of the builds passes tests, the other fails in production. The bug is not in the code — it is in the environment. Nobody can reproduce it.

Failure mode 3: The Friday 5 PM push

A developer pushes code at 5 PM on Friday. They did not run the tests because they were in a rush. The code has a bug. Nobody notices until Monday morning when a customer reports an outage. The developer has forgotten exactly what they changed. Debugging takes four hours.

Failure mode 4: The wrong-environment deploy

You mean to deploy to the staging environment. You type the wrong command and deploy to production instead. The new code has a database migration that breaks existing records. There is no easy undo.

CI/CD eliminates all four failure modes with one simple rule: the only way to change what is running in production is to push code to a specific Git branch. Everything else — running tests, building images, scanning for vulnerabilities, pushing to ECR, telling Kubernetes to update — happens automatically, the same way, every single time.

1.2 — CI vs CD: Three Terms That Sound the Same

You will see three terms used constantly. They mean different things and it matters:

Continuous Integration (CI)

Every code change is automatically tested the moment it is pushed. The goal is to catch bugs within minutes, not days.

When a developer pushes a commit, an automated server pulls the code, installs dependencies, and runs the test suite. If any test fails, the developer gets a red X on their commit in GitHub and can fix it immediately — before anyone else on the team pulls the broken code.

CI is about quality gates for incoming code. It does not touch production. It only says: is this code safe to consider for deployment?

Continuous Delivery

After CI passes, the code is automatically built, packaged (Docker image built and pushed), and made ready to deploy. But a human still has to approve the deploy.

Think of Continuous Delivery as: 'the button is built, ready, and armed. Someone must still press it.' This is the pattern for production deployments in most companies. You want fast, automated builds, but you want a human checking that now is a good time to deploy (e.g., not during a traffic spike, not right before a holiday, not without notifying the on-call engineer).

Continuous Deployment

After CI passes, the code is automatically deployed with no human involvement. Code goes from git push to running in front of real users in minutes, with no button to press.

This is appropriate for development and staging environments where the cost of a bad deploy is low. It is also used in production by some companies with mature testing practices, feature flags, and fast rollback mechanisms. But it requires high confidence in your test suite.

In This Lab	We Use
Dev environment	Continuous Deployment — push to the dev branch auto-deploys with no approval.
Prod environment	Continuous Delivery — push to the main branch runs the full build, but pauses and waits for a human to approve the production deploy.

1.3 — What Is a Pipeline?

A pipeline is a sequence of automated stages that run when you push code. Each stage is a set of related tasks. Stages run one after another. If a stage fails, the pipeline stops — broken code never reaches the next stage.

Here is the pipeline you will build in this lab, laid out stage by stage:

```
Source  →  Test  →  Build  →  Scan  →  Push  →  Deploy
(push)   (mocha) (docker) (trivy) (ECR)  (kubectl)

Stage 1 (Source):  Developer pushes a commit to GitHub.
Stage 2 (Test):    Pipeline runs unit tests and integration tests.
                   If any test fails: STOP. Notify developer.
Stage 3 (Build):   Pipeline builds Docker images for frontend and backend.
                   If build fails (syntax error, missing dependency): STOP.
Stage 4 (Scan):    Trivy scans images for known vulnerabilities.
                   If CRITICAL or HIGH CVE found: STOP. Do not push.
Stage 5 (Push):    Images pushed to Amazon ECR with a unique tag.
Stage 6 (Deploy):  Pipeline tells Kubernetes to update deployments.
                   For dev: automatic. For prod: waits for approval.
```

Why put stages in this specific order? Because the cost of running each stage differs dramatically:

Stage	Typical Time
Test (unit + integration)	30 seconds to 2 minutes
Build (docker build for 2 images)	1 to 3 minutes
Scan (Trivy on 2 images)	20 to 60 seconds
Push (docker push to ECR)	30 seconds to 2 minutes
Deploy (kubectl rollout)	30 seconds to 2 minutes

If a unit test will fail, we want to know in 30 seconds, not after a 5-minute build and 2-minute push. Putting the cheapest, fastest checks first is called 'failing fast.' It saves time for developers and money for the company (CI runners cost per minute).

1.4 — GitHub Actions: The Tool We Use

There are many CI/CD platforms — Jenkins, CircleCI, GitLab CI, Travis, TeamCity, AWS CodePipeline, and dozens more. We use GitHub Actions because it is built into GitHub, requires no separate signup or billing, and has a generous free tier for public and private repositories.

GitHub Actions executes pipelines defined in YAML files that live in your repository under the path .github/workflows/. Each YAML file is called a workflow. You can have many workflows in one repo — for example, ci.yml for the main pipeline and terraform.yml for infrastructure changes.

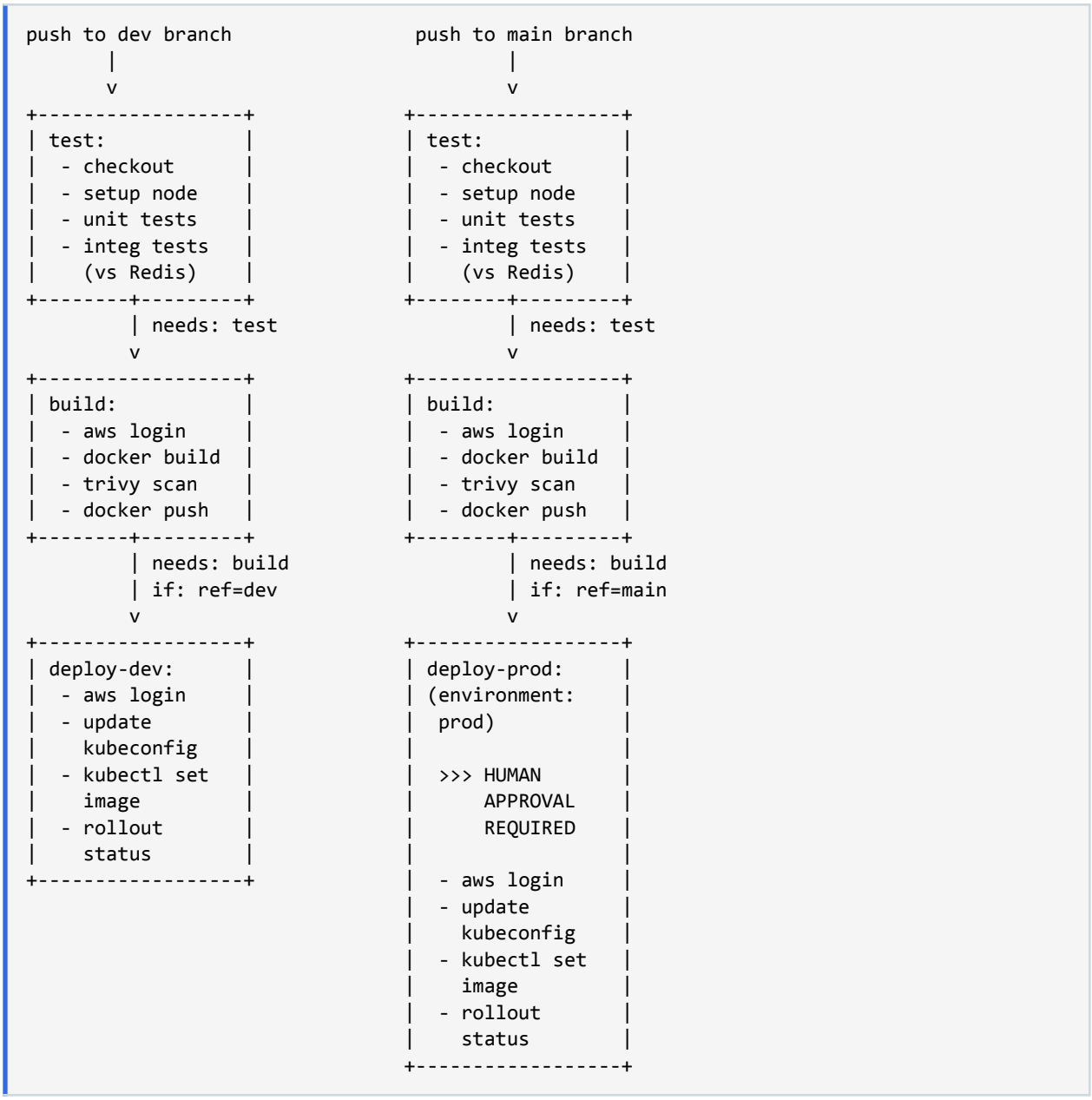
1.5 — GitHub Actions Vocabulary

Before writing any YAML, you need to know what the words mean. You will see each of these terms dozens of times in the lab, and this table is worth bookmarking.

Term	Meaning
Workflow	A single YAML file in .github/workflows/. Defines one pipeline. A workflow has a name, a trigger, and one or more jobs.
Event (Trigger)	What causes a workflow to run. Common events: push (when code is pushed), pull_request (when a PR is opened or updated), workflow_dispatch (a manual button in the UI), schedule (cron job).
Job	A group of steps that run on one virtual machine. A workflow contains one or more jobs. By default, jobs run in parallel. Use needs: to make one job wait for another.
Step	A single task within a job. Steps run sequentially. A step is either a shell command (using run:) or a call to a reusable Action (using uses:).
Action	A reusable, pre-built unit of logic. Hosted on GitHub. Version-pinned with @. Examples: actions/checkout@v4 clones your repo, aws-actions/configure-aws-credentials@v4 authenticates to AWS.
Runner	The virtual machine that executes a job. GitHub provides free runners running Ubuntu, Windows, or macOS. Each job gets a fresh runner that is destroyed when the job ends — anything you write to disk is lost between jobs.
Secret	An encrypted variable stored in GitHub's settings. Used for sensitive values like AWS credentials. Referenced in YAML as ${{ secrets.NAME }}. GitHub automatically masks secret values in logs.
Variable (Context)	A built-in value that GitHub provides about the workflow run. Examples: github.sha (commit hash), github.ref (branch), github.actor (who pushed).
Environment	A named deployment target (like dev, staging, prod). Can have its own secrets AND protection rules — for example, 'require approval from a specific reviewer.' Our prod environment uses this for manual approval.

Term	Meaning
Artifact	A file produced by one job that can be shared with another job or downloaded after the workflow finishes. We do not use artifacts in this lab, but they are important for larger pipelines.
Matrix	A way to run the same job with multiple different inputs in parallel. For example, test against Node 18 AND Node 20 simultaneously. Not used in this lab but worth knowing about.

1.6 — The Pipeline You Will Build



Part 2 — Infrastructure Setup

The CI/CD pipeline updates infrastructure that already exists. Before we can write any pipeline code, that infrastructure must be in place. This part walks you through creating it. Every step has a reference back to Lab 009 if you want a deeper explanation — here we focus on what you need to do.

Step 1 — Create the EKS Cluster

EKS (Elastic Kubernetes Service) is AWS's managed Kubernetes. You provide worker nodes (EC2 instances); AWS manages the control plane. The eksctl tool automates cluster creation by provisioning a VPC, subnets, security groups, IAM roles, the control plane, and a node group — all in one command.

Run this command:

```
eksctl create cluster \
  --name cicd-lab \
  --region us-east-1 \
  --nodegroup-name workers \
  --node-type t3.medium \
  --nodes 2 \
  --nodes-min 1 \
  --nodes-max 3 \
  --managed
```

Here is what each flag does:

Flag	What It Does
--name cicd-lab	The cluster name. Used later when you run aws eks update-kubeconfig or eksctl delete cluster.
--region us-east-1	Which AWS region to create the cluster in. Stick with one region throughout the lab — mixing regions will cause ECR image pull failures.
--nodegroup-name workers	Name of the EC2 Auto Scaling Group that holds your worker nodes.
--node-type t3.medium	Instance type for workers. t3.medium has 2 vCPU and 4 GB RAM — enough for this lab's pods. NOT free tier.
--nodes 2	Initial number of worker nodes. Two is the minimum for realistic multi-node testing.

Flag	What It Does
--nodes-min 1 / --nodes-max 3	Auto-scaling bounds. The cluster can scale between 1 and 3 nodes if you enable the Cluster Autoscaler (we don't in this lab).
--managed	Use a managed node group. AWS handles node updates and replacement. Much easier than self-managed node groups.

This command takes 15 to 20 minutes. It does a lot:

- Creates a new VPC with public and private subnets across multiple Availability Zones
- Creates security groups for the control plane and nodes
- Provisions the EKS control plane (managed by AWS)
- Creates IAM roles for the control plane and for worker nodes
- Launches the EC2 instances and joins them to the cluster
- Configures kubectl on your local machine to talk to the new cluster

When it finishes, verify by running:

```
kubectl get nodes
# NAME                              STATUS   ROLES    AGE   VERSION
# ip-192-168-12-34.ec2.internal     Ready    <none>   5m    v1.28.x
# ip-192-168-56-78.ec2.internal     Ready    <none>   5m    v1.28.x
```

Both nodes should show STATUS = Ready. If one shows NotReady, wait another minute and re-run — nodes take a moment to fully register.

> ⚠️ **Warning**
> The moment eksctl create cluster succeeds, billing starts at roughly $0.18/hour ($0.10 control plane + $0.08 two t3.medium nodes). The meter is now running.

Step 2 — Create ECR Repositories

ECR (Elastic Container Registry) is AWS's Docker image registry. Your Kubernetes pods will pull images from ECR. You need one repository per image — we need two (frontend and backend).

```
aws ecr create-repository --repository-name k8s-frontend --region us-east-1
aws ecr create-repository --repository-name k8s-backend  --region us-east-1
```

What each part does:

- `aws ecr create-repository` — API call to create a new repository in ECR.
- `--repository-name` — the name of the repo. Becomes part of the image URL.

- `--region` — which region to create the repo in. MUST match your EKS cluster region.

Verify:

```
aws ecr describe-repositories \
  --region us-east-1 \
  --query 'repositories[].repositoryName' \
  --output text
# Expected output:  k8s-backend   k8s-frontend
```

> 🔍 **Deep Dive**
>
> An ECR image URL has this structure:
>
> <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/<REPO>:<TAG>
>
> For example: 123456789012.dkr.ecr.us-east-1.amazonaws.com/k8s-backend:v1
>
> The account ID makes the URL unique globally. That is why ECR images cannot be shared across accounts without explicit permission grants.
>
> The tag is the version label. A repository holds many tagged versions of the same logical image. In this lab we will tag images with Git commit SHAs (like 'a3f9c21b'), which gives us a unique tag for every build.

Step 3 — Create Two Kubernetes Namespaces

A namespace is a virtual partition inside a Kubernetes cluster. Resources in one namespace cannot see resources in another. We use two namespaces to run dev and prod copies of the same application on the same cluster without letting them interfere with each other.

```
kubectl create namespace dev
kubectl create namespace prod

kubectl get namespaces
```

You should see dev and prod in the list, alongside system namespaces like default, kube-system, and kube-public.

> 🔍 **Deep Dive**
>
> Why not two separate clusters for dev and prod? In a real production setup, you would. Total isolation is safer — a mistake in dev can never reach prod. But each EKS cluster costs $72/month for the control plane alone. Two clusters = $144/month before a single workload runs.
>
> For a learning lab, two namespaces in one cluster teaches the same patterns (separate environments, separate secrets, separate access control) at 1/2 the cost. In a real job, you would typically use at least separate node groups and often separate clusters for prod.

Step 4 — Create the Local Project Structure

Create a new local folder. Do not reuse the Lab 009 folder — we will write new manifests that are subtly different (explained in Part 4), and mixing them leads to confusion.

```
mkdir cicd-lab && cd cicd-lab
mkdir -p app/frontend manifests .github/workflows terraform
```

The final structure after all parts of this lab:

```
cicd-lab/
├── .github/
│   └── workflows/
│       ├── ci.yml              (main pipeline — Parts 6-9)
│       └── terraform.yml       (Terraform pipeline — Part 11)
├── app/
│   ├── server.js               (Node.js backend)
│   ├── package.json            (Node dependencies + test scripts)
│   ├── server.test.js          (unit test)
│   ├── server.integration.test.js  (Redis integration test)
│   ├── Dockerfile              (backend image build)
│   └── frontend/
│       ├── index.html
│       ├── nginx.conf
│       └── Dockerfile          (frontend image build)
├── manifests/
│   ├── redis.yaml
│   ├── backend.yaml
│   └── frontend.yaml
└── terraform/
    ├── versions.tf
    ├── variables.tf
    ├── main.tf
    └── terraform.tfvars
```

> 📝 **Note**
> The dot-prefix on .github matters. GitHub specifically looks for workflow YAML files at the exact path .github/workflows/. If you accidentally put them in github/workflows/ (no dot) or workflows/ (top-level), GitHub will not find them and the pipeline will never run.

Part 3 — Application Code

We reuse the three-service application from Lab 009: an nginx frontend, a Node.js backend, and a Redis data store. The code is mostly the same, but we add two test files and adjust the Dockerfile so test dependencies are not shipped to production.

Step 5 — Backend Source Code

Create app/server.js. This is a small Express server with four API endpoints, all backed by Redis:

```
const express = require("express");
const Redis = require("ioredis");
const app = express();
app.use(express.json());

const redis = new Redis({
  host: process.env.REDIS_HOST || 'localhost',
  port: 6379,
});

app.get("/api/health", async (req, res) => {
  const pong = await redis.ping();
  res.json({ status: "ok", redis: pong, pod: process.env.HOSTNAME });
});

app.get("/api/visits", async (req, res) => {
  const count = await redis.incr("visits");
  res.json({ visits: count, pod: process.env.HOSTNAME });
});

app.get("/api/messages", async (req, res) => {
  const msgs = await redis.lrange("messages", 0, -1);
  res.json(msgs.map(m => JSON.parse(m)));
});

app.post("/api/messages", async (req, res) => {
  const msg = {
    text: req.body.text,
    pod: process.env.HOSTNAME,
    time: new Date().toISOString()
  };
  await redis.lpush("messages", JSON.stringify(msg));
  res.status(201).json(msg);
});

app.listen(3000, () => console.log('Backend running on :3000'));
```

Key points about this code:

- `process.env.REDIS_HOST || 'localhost'` — reads the REDIS_HOST environment variable, defaulting to localhost. In Kubernetes, the deployment sets

REDIS_HOST=redis-service (the K8s DNS name of the Redis service). In the CI integration test, the workflow sets REDIS_HOST=localhost (the service container is on localhost:6379).

- `process.env.HOSTNAME` — Kubernetes sets HOSTNAME to the pod name. We return it in every response so you can see which pod served each request — useful for proving load balancing across replicas.
- The /api/visits endpoint uses `redis.incr` which atomically increments a counter. If two requests hit at the same moment, each still gets a unique incremented value.

Step 6 — package.json with Test Scripts

Create app/package.json:

```
{
  "name": "k8s-backend",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.0",
    "ioredis": "^5.3.0"
  },
  "devDependencies": {
    "mocha": "^10.2.0"
  },
  "scripts": {
    "start": "node server.js",
    "test": "mocha server.test.js --exit",
    "test:integration": "mocha server.integration.test.js --exit --timeout 10000"
  }
}
```

What each section does:

Field	Explanation
dependencies	Packages needed at runtime. express for the HTTP server, ioredis for talking to Redis. These are installed in production images.
devDependencies	Packages needed only for development and testing. mocha is the test runner. These are NOT installed in production images (we pass --omit=dev in the Dockerfile).
scripts.start	What runs when Kubernetes starts the container (our Dockerfile's CMD calls 'npm start').
scripts.test	Runs the unit test file. --exit forces Mocha to exit after tests complete, otherwise it hangs waiting for open connections.

Field	Explanation
scripts.test:integration	Runs the integration test. --timeout 10000 gives each test up to 10 seconds (network calls to Redis are slower than pure-logic unit tests).
^4.18.0 (caret)	Semver: accept any 4.x.y version, but not 5.x. Lets you get patch and minor updates automatically

🔍 Deep Dive

Why separate 'test' and 'test:integration' scripts instead of one combined script?
Two reasons. First, unit tests need no external dependencies, so they can run anywhere — on a laptop, in any CI environment — with zero setup. Integration tests need a real Redis server, which is extra setup cost.

Second, failure modes are different. A failing unit test means the code logic is wrong. A failing integration test could mean the code is wrong, OR Redis is unreachable, OR the Redis connection timed out. Keeping them separate makes debugging easier.
In the pipeline, we run unit tests first (fail fast — 5 seconds), then integration tests (30+ seconds with Redis startup). If unit tests fail, we never even start Redis.

Step 7 — Backend Dockerfile

Create app/Dockerfile:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY server.js .
EXPOSE 3000
CMD ["npm", "start"]
```

Line-by-line explanation:

Line	Explanation
FROM node:18-alpine	Base image. Node 18 on Alpine Linux (tiny, ~50 MB). Smaller base images = faster pulls in production.
WORKDIR /app	All subsequent commands run inside /app. Equivalent to 'cd /app'. The directory is created if it does not exist.

Line	Explanation
COPY package*.json ./	Copy package.json AND package-lock.json (if present). Copying these two files first is a Docker best practice — it lets Docker cache the npm install step if dependencies have not changed.
RUN npm ci --omit=dev	Install dependencies. npm ci is faster and stricter than npm install. --omit=dev skips devDependencies (mocha), so test code and test libraries are not shipped to production.
COPY server.js .	Copy the application code AFTER installing dependencies. If you change server.js but not package.json, Docker reuses the cached dependency layer — much faster builds.
EXPOSE 3000	Documentation that the container listens on port 3000. Does NOT actually publish the port — kubectl or docker -p does that. EXPOSE is informational.
CMD ["npm", "start"]	Default command when the container starts. Uses the 'exec' form (JSON array), which is preferred over shell form ('npm start') because it handles signals correctly — important for graceful shutdown in Kubernetes.

Step 8 — Test Files

Unit test: app/server.test.js

A unit test checks code logic in isolation, without external dependencies. Our unit test is intentionally trivial — the point is to verify the pipeline runs tests, not to write a deep test suite:

```
const assert = require("assert");

describe("Health Check", () => {
  it("should return ok status", () => {
    const status = "ok";
    assert.strictEqual(status, "ok");
  });
});
```

Structure breakdown:

- `describe(...)` — a Mocha grouping. All tests inside describe are related and appear together in the output.
- `it(...)` — a single test case. The string should describe what the test verifies.

- `assert.strictEqual(a, b)` — the Node built-in assertion. Checks that a === b. If false, the test fails and Mocha prints a diff.

Integration test: app/server.integration.test.js

Integration tests run your code against real external services — in our case, a real Redis server:

```
const assert = require("assert");
const Redis = require("ioredis");

const redis = new Redis({
  host: process.env.REDIS_HOST || 'localhost',
  port: 6379,
});

describe("Redis Integration", () => {
  after(() => redis.quit());

  it("should connect and set/get a value", async () => {
    await redis.set("test-key", "hello-ci");
    const value = await redis.get("test-key");
    assert.strictEqual(value, "hello-ci");
  });

  it("should increment a counter", async () => {
    await redis.del("counter");
    await redis.incr("counter");
    await redis.incr("counter");
    const count = await redis.get("counter");
    assert.strictEqual(count, "2");
  });

  it("should handle list operations", async () => {
    await redis.del("test-messages");
    await redis.lpush("test-messages", JSON.stringify({ text: "hello" }));
    const msgs = await redis.lrange("test-messages", 0, -1);
    assert.strictEqual(msgs.length, 1);
    assert.strictEqual(JSON.parse(msgs[0]).text, "hello");
  });
});
```

Each test mirrors one of the real application endpoints:

- **set/get** — mimics how /api/health pings Redis.
- **increment counter** — mimics how /api/visits increments the visit counter.
- **list operations** — mimics how /api/messages pushes and reads messages.

The after(() => redis.quit()) hook closes the Redis connection after all tests in the suite finish. Without it, Mocha hangs because the connection keeps the Node process alive.

Step 9 — Frontend Files

Create app/frontend/index.html. This file will be served by nginx. Replace [YOUR NAME] with your actual name — this must be visible in the rendered page for submission screenshots:

```
<!DOCTYPE html>
<html>
<head>
  <title>CI/CD Lab</title>
  <style>
    body { font-family: Arial; max-width: 700px; margin: 40px auto; padding: 20px; }
    .card { background: #f5f5f5; border-radius: 8px; padding: 16px; margin: 12px 0; }
    button { background: #326CE5; color: white; border: none; padding: 10px 20px;
             border-radius: 4px; cursor: pointer; font-size: 14px; }
    input { padding: 10px; width: 300px; border: 1px solid #ddd; border-radius: 4px; }
  </style>
</head>
<body>
  <h1>CI/CD Lab — [YOUR NAME]</h1>
  <div class="card" id="health">Loading health...</div>
  <div class="card" id="visits">Loading visits...</div>
  <div class="card">
    <input id="msg" placeholder="Type a message">
    <button onclick="send()">Send</button>
  </div>
  <div id="messages"></div>
  <script>
    const api = '/api';
    fetch(api+'/health').then(r=>r.json()).then(d=>{
      document.getElementById('health').innerHTML =
        '<b>Health:</b> '+d.status+' | <b>Redis:</b> '+d.redis+' | <b>Pod:</b> '+d.pod;
    });
    fetch(api+'/visits').then(r=>r.json()).then(d=>{
      document.getElementById('visits').innerHTML =
        '<b>Total visits:</b> '+d.visits+' | <b>Served by pod:</b> '+d.pod;
    });
    function load(){
      fetch(api+'/messages').then(r=>r.json()).then(msgs=>{
        document.getElementById('messages').innerHTML = msgs.map(m =>
          '<div class="card"><b>'+m.pod+'</b>: '+m.text+'
<small>('+m.time+')</small></div>'
        ).join('');
      });
    }
    function send(){
      fetch(api+'/messages',{
        method:'POST',
        headers:{'Content-Type':'application/json'},
        body:JSON.stringify({text:document.getElementById('msg').value})
      }).then(()=>{ document.getElementById('msg').value=''; load(); });
    }
    load();
  </script>
</body>
</html>
```

Create app/frontend/nginx.conf. This is the critical reverse-proxy configuration:

```
server {
  listen 80;
  root /usr/share/nginx/html;
  location / { try_files $uri $uri/ /index.html; }
  location /api/ {
    proxy_pass http://backend-service:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
  }
}
```

What this nginx config does:

Directive	Meaning
listen 80	Accept HTTP connections on port 80.
root /usr/share/nginx/html	Serve static files from this directory (where our Dockerfile puts index.html).
location / { try_files ... }	For any request NOT matching /api/, try to serve a static file. Falls back to index.html if not found (for SPA routing).
location /api/ { proxy_pass ... }	For requests to /api/*, forward them to the backend. backend-service is the K8s DNS name of the backend ClusterIP service. Kubernetes resolves it automatically inside the cluster.
proxy_set_header Host $host	Forward the original Host header so the backend sees the real hostname the user visited.
proxy_set_header X-Real-IP $remote_addr	Forward the user's IP so the backend can log it.

Create app/frontend/Dockerfile:

```
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY index.html /usr/share/nginx/html/
EXPOSE 80
```

No CMD needed — the nginx base image already has one that starts nginx in the foreground.

Step 10 — Install Dependencies and Run Tests Locally

Before pushing to GitHub, make sure tests pass locally. This saves you from a round-trip to the CI system to catch trivial errors:

```
cd app
npm install       # installs everything including mocha
npm test          # should print: passing (Xms)
cd ..
```

> 📝 **Note**
>
> Do not run the integration test locally yet — it requires a Redis server that we have not started. The CI pipeline will start one automatically using a service container.
>
> If you want to run it locally for your own sanity check: docker run -d -p 6379:6379 redis:7-alpine, then cd app && npm run test:integration, then docker ps and docker stop <id> to clean up the Redis container.

Part 4 — Kubernetes Manifests

A Kubernetes manifest is a YAML file that describes a resource — a Deployment, a Service, an Ingress. You give manifests to Kubernetes via kubectl apply, and Kubernetes makes the cluster match the manifest.

4.1 — Why These Manifests Differ From Lab 009

In Lab 009, every manifest had this in its metadata section:

```
metadata:
  name: backend
  namespace: k8s-lab     <-- hardcoded namespace
spec:
  ...
```

This was fine for Lab 009 because everything lived in one namespace. In Lab 010, we want the SAME manifests to deploy to TWO different namespaces (dev and prod). The pipeline should decide at apply time which namespace to target.

If we leave the hardcoded namespace: k8s-lab, kubectl apply -f manifests/ -n dev is silently ignored. The -n flag is overridden by the namespace field inside the manifest. Every resource ends up in k8s-lab regardless of the -n flag.

The fix: remove the namespace: line from every manifest. Then:

- kubectl apply -n dev -f manifests/ deploys everything to the dev namespace.
- kubectl apply -n prod -f manifests/ deploys everything to the prod namespace.

- Same manifest, two different deployments, chosen at apply time. This is exactly what we need.

> ⚠️ **Warning**
> This is the single most common bug students hit in this lab. Before applying any manifest, grep it to double-check:
>
> grep -r 'namespace:' manifests/
>
> If this returns anything, remove those lines. The -n flag must win.

Step 11 — Write the Three Manifests

manifests/redis.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
spec:
  replicas: 1
  selector:
    matchLabels: { app: redis }
  template:
    metadata:
      labels: { app: redis }
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          ports:
            - containerPort: 6379
---
apiVersion: v1
kind: Service
metadata:
  name: redis-service
spec:
  type: ClusterIP
  selector: { app: redis }
  ports:
    - port: 6379
      targetPort: 6379
```

Two resources in one file, separated by ---. The Deployment creates and maintains one Redis pod. The Service gives it a stable DNS name (redis-service) reachable from other pods in the namespace. Type ClusterIP means internal-only — Redis is never reachable from outside the cluster.

manifests/backend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 2
  selector:
    matchLabels: { app: backend }
  template:
    metadata:
      labels: { app: backend }
    spec:
      containers:
        - name: backend
          image: PLACEHOLDER
          ports:
            - containerPort: 3000
          env:
            - name: REDIS_HOST
              value: redis-service
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits:   { cpu: "500m", memory: "256Mi" }
          livenessProbe:
            httpGet: { path: /api/health, port: 3000 }
            initialDelaySeconds: 10
            periodSeconds: 15
          readinessProbe:
            httpGet: { path: /api/health, port: 3000 }
            initialDelaySeconds: 5
            periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  type: ClusterIP
  selector: { app: backend }
  ports:
    - port: 3000
      targetPort: 3000
```

Key features of the backend manifest:

Field	What It Does
replicas: 2	Run 2 identical backend pods. Traffic is load-balanced across them by the Service.

Field	What It Does
image: PLACEHOLDER	Intentional placeholder. The pipeline will overwrite this with kubectl set image. We cannot put the real image URL here because that URL contains the AWS account ID, which we do not want hardcoded in the repo.
env: REDIS_HOST=redis-service	Tell the backend where to find Redis. 'redis-service' is the K8s DNS name of the Service defined in redis.yaml.
resources.requests	The pod needs at least 100 millicores (0.1 CPU) and 128 MiB RAM. Kubernetes won't schedule the pod on a node that doesn't have this available.
resources.limits	The pod can use up to 500 millicores and 256 MiB. Memory over limit → pod is killed (OOMKilled). CPU over limit → throttled.
livenessProbe	Kubernetes hits /api/health every 15s. If it fails 3 times, K8s kills the pod and creates a new one. This catches hung containers that are still 'running' but not serving traffic.
readinessProbe	Kubernetes hits /api/health before sending traffic. If it fails, the pod is removed from the Service's endpoint list until it recovers. This prevents sending traffic to a pod that is still starting up.

manifests/frontend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 2
  selector:
    matchLabels: { app: frontend }
  template:
    metadata:
      labels: { app: frontend }
    spec:
      containers:
        - name: frontend
          image: PLACEHOLDER
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
```

```
metadata:
  name: frontend-service
spec:
  type: ClusterIP
  selector: { app: frontend }
  ports:
    - port: 80
      targetPort: 80
```

Simpler than the backend — no probes (we're keeping it minimal), no env vars. Note that the frontend Service is also ClusterIP, not LoadBalancer. For this lab we access the frontend via kubectl port-forward. In a real deployment you would add an Ingress to expose it to the internet (see Lab 009 for that pattern).

4.2 — First-Time Manual Deploy (Seeding the Cluster)

The pipeline uses kubectl set image deployment/backend <image> to update an existing deployment. If no deployment exists yet, that command fails with 'deployments.apps "backend" not found'.

We need to seed both namespaces with an initial version of the app so the pipeline has something to update. This is a one-time manual step.

Step 12 — Build and Push Initial v1 Images

First, capture your account ID and region in shell variables for the rest of the lab:

```
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export REGION=us-east-1
echo "Account: $ACCOUNT_ID"
echo "Region: $REGION"
```

Authenticate Docker to ECR. The auth token expires after 12 hours, so you may need to re-run this later:

```
aws ecr get-login-password --region $REGION | \
  docker login \
    --username AWS \
    --password-stdin \
    $ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com
```

🔍 **Deep Dive**

Why this convoluted login command? ECR does not use username/password authentication. It uses a temporary token issued by the AWS API. aws ecr get-login-password outputs that token. We pipe it into docker login --password-stdin, which reads the password from stdin instead of a command-line argument.

Why stdin? Passing passwords on the command line (docker login -p <password>) exposes them in process listings (ps aux) and shell history. --password-stdin is the secure alternative. Every time you see a shell token piped into a login command, this is why.

Build and push the backend image with tag v1:

```
docker build \
  -t $ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/k8s-backend:v1 \
  ./app

docker push $ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/k8s-backend:v1
```

Then the frontend:

```
docker build \
  -t $ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/k8s-frontend:v1 \
  ./app/frontend

docker push $ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/k8s-frontend:v1
```

Step 13 — Deploy the Initial Version to Both Namespaces

Apply the manifests and then immediately replace the PLACEHOLDER image with the real v1 image:

📝 **Note**

Make sure $ACCOUNT_ID and $REGION are still set in your shell from Step 12. If you opened a new terminal, re-run the export lines before continuing.

```
# --- DEV ---
kubectl -n dev apply -f manifests/
kubectl -n dev set image deployment/backend
backend=$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/k8s-backend:v1
kubectl -n dev set image deployment/frontend
frontend=$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/k8s-frontend:v1

# --- PROD ---
kubectl -n prod apply -f manifests/
kubectl -n prod set image deployment/backend
backend=$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/k8s-backend:v1
kubectl -n prod set image deployment/frontend
frontend=$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/k8s-frontend:v1
```

Between the apply and the set image, pods will briefly try to pull image 'PLACEHOLDER' from Docker Hub and fail with ImagePullBackOff. This is expected. set image triggers a new rollout with the real image and the failing pods are replaced within seconds.

Wait for rollouts to finish and verify:

```
kubectl -n dev  rollout status deployment/backend  --timeout=120s
kubectl -n dev  rollout status deployment/frontend --timeout=120s
kubectl -n prod rollout status deployment/backend  --timeout=120s
kubectl -n prod rollout status deployment/frontend --timeout=120s

kubectl get pods -n dev
kubectl get pods -n prod
# Each namespace should show 5 pods Running:
#   1 redis + 2 backend + 2 frontend
```

✅ **Tip**

If pods stay in ImagePullBackOff after the set image, the EKS node group's IAM role probably does not have ECR read permissions. Check by running:

aws iam list-attached-role-policies --role-name $(aws eks describe-nodegroup --cluster-name cicd-lab --nodegroup-name workers --query 'nodegroup.nodeRole' --output text | awk -F/ '{print $2}')

You should see AmazonEC2ContainerRegistryReadOnly in the list. eksctl usually attaches this automatically.

Part 5 — GitHub Setup

GitHub will host your code, run your pipeline, store your secrets, and enforce branch protection. This part covers all the GitHub-side setup.

Step 14 — Create the GitHub Repository

Go to github.com and sign in. Click the + icon in the top-right, then New repository. Fill in:

- **Owner:** your username.
- **Repository name:** `cicd-lab`.
- **Visibility:** Private (you do not want your AWS account ID in a public repo).
- **Initialize:** leave all checkboxes UNCHECKED. We will push existing code — if GitHub creates a README, there will be merge conflicts.

Click Create repository. You will see a page with instructions. We use the 'push an existing repository from the command line' option.

Step 15 — Initialize Git and Push Your Code

From inside your cicd-lab folder:

```
git init
git add .
git commit -m "Initial commit: application code and K8s manifests"
git branch -M main
git remote add origin https://github.com/<YOUR-USERNAME>/cicd-lab.git
git push -u origin main
```

What each command does:

Command	Purpose
git init	Creates a new Git repository in the current folder (adds a hidden .git/ directory).
git add .	Stage all files in the current directory for commit.
git commit -m "..."	Save the staged files as a commit with the given message.
git branch -M main	Rename the default branch to 'main'. Older Git versions used 'master'. GitHub now prefers 'main'.
git remote add origin <URL>	Link your local repo to the GitHub repo. 'origin' is the conventional name for the primary remote.
git push -u origin main	Upload your commits to GitHub. -u sets 'origin main' as the default so future 'git push' works with no arguments.

Step 16 — Create the dev Branch

The pipeline behaves differently depending on which branch you push to. main is production; dev is development. We need both branches to exist on GitHub.

```
git checkout -b dev
git push -u origin dev
```

git checkout -b dev creates a new branch named 'dev' and switches to it in one command. The push then uploads it to GitHub and sets the tracking relationship.

You now have two branches:

- **main** — production code. Pushes here will eventually trigger the prod pipeline (with manual approval).
- **dev** — development code. Pushes here will trigger the dev pipeline (no approval).

Step 17 — Add GitHub Secrets

The pipeline needs AWS credentials to push to ECR and update the EKS cluster. Hardcoding credentials in the YAML file would be catastrophic — anyone who cloned the repo could use them. GitHub Secrets solve this.

How GitHub Secrets work

Secrets are encrypted values stored in GitHub. They are decrypted only when a workflow runs, injected as environment variables, and never printed in logs. If a secret value accidentally appears in any output (for example, an echo command), GitHub replaces it with *** before showing the log.

Navigate to your repo → Settings → Secrets and variables → Actions → New repository secret. Create each of these, one at a time:

Secret Name	Value	Purpose
AWS_ACCESS_KEY_ID	Your IAM access key (starts with AKIA…)	Authenticates to AWS APIs
AWS_SECRET_ACCESS_KEY	Your IAM secret (40+ chars)	Paired with the access key
AWS_REGION	us-east-1 (or whichever region you used)	Targets the right region
AWS_ACCOUNT_ID	Your 12-digit AWS account ID	Constructs ECR image URIs
EKS_CLUSTER_NAME	cicd-lab	Used by aws eks update-kubeconfig

Creating the IAM credentials

If you do not already have IAM access keys for this lab, create a dedicated IAM user:

22. AWS Console → IAM → Users → Create user.
23. **Name:** `cicd-lab-github`.
24. Set permissions: attach existing policies directly → AdministratorAccess (for lab simplicity).
25. After user is created, click Security credentials → Create access key → Command Line Interface.
26. Copy both the Access key ID and Secret access key. The secret is shown only ONCE.

> ⚠️ **Warning**
> AdministratorAccess is dangerous. A leaked key with AdministratorAccess gives the attacker full control of your AWS account — they can mine crypto, delete your data, launch

expensive resources. NEVER use AdministratorAccess for anything other than a short-lived lab. Delete this IAM user when you finish the lab.

For production systems, create a narrowly-scoped policy that allows only what the pipeline needs: ecr:*, eks:DescribeCluster, and maybe one or two others. Better yet, use OIDC federation (see the Tip below).

Step 18 — Create GitHub Environments

GitHub Environments are named deployment targets with their own protection rules. We need two: dev and prod.

Navigate to Settings → Environments → New environment.

Create 'dev' environment

27. **Name:** `dev`.
28. Leave all protection rules UNCHECKED.
29. Click Save protection rules (or just Configure environment — the exact button label varies).

Create 'prod' environment

30. **Name:** `prod`.
31. **Check Required reviewers.** Add yourself as a reviewer.
32. Leave wait timer blank, or set it to 0 minutes.
33. Save the configuration.

The required-reviewer rule on prod is the manual-approval gate. Any pipeline job declared with environment: prod will pause in a Waiting state until you click Approve and deploy.

🔍 **Deep Dive**

Why use environments at all? Couldn't we just use an if: condition in the workflow?

You could, but environments give you three things conditions alone don't:

1. Required reviewers — force a human to click Approve before a job runs. Conditions can't do this.

2. Environment-scoped secrets — you can set AWS_ACCESS_KEY_ID differently for dev vs prod. Jobs with environment: dev get the dev key; jobs with environment: prod get the prod key. This lets you follow least-privilege in production without maintaining two separate workflows.

3. Deployment history — GitHub shows a log of 'who deployed what, when' per environment. The Deployments tab in your repo tracks this automatically for any job that uses environment:.

Part 6 — YAML Deep Dive and Your First Workflow

Before we write our workflow, take a moment to understand YAML properly. Most CI/CD errors are YAML errors — indentation wrong, colon missing, quotes where they shouldn't be. Five minutes of YAML fluency saves hours of debugging.

6.1 — YAML in 10 Minutes

YAML (YAML Ain't Markup Language) is a data format designed to be human-readable. It represents three things: scalars (strings, numbers, booleans), lists (ordered collections), and maps (key-value pairs).

Scalars

```
# Strings (quotes are usually optional)
name: CI Pipeline
description: "Runs tests and deploys"
path: ./app                      # no quotes needed for simple strings

# Numbers and booleans
replicas: 3
enabled: true
failures: 0

# Multi-line strings — two variants:
# Literal block (preserves newlines)
script: |
  echo "first line"
  echo "second line"

# Folded block (collapses newlines to spaces)
description: >
  This becomes
  one single line.
```

Lists

```
# Block style (preferred — more readable)
branches:
  - main
  - dev
  - staging

# Flow style (compact — use for short lists)
branches: [main, dev, staging]
```

Maps

```
# Block style
server:
  host: localhost
  port: 3000
```

```
  name: backend

# Flow style (one line)
server: { host: localhost, port: 3000, name: backend }
```

Nested structures

```
jobs:                    # map
  test:                  #   -> map
    runs-on: ubuntu-latest #      -> scalar
    steps:               #      -> list of maps
      - name: Checkout
        uses: actions/checkout@v4
      - name: Run tests
        run: npm test
```

Comments

Comments start with # and go to end of line. YAML has no multi-line comments — each comment line needs its own #.

> ⚠️ **Warning**
>
> INDENTATION RULES THAT MATTER:
>
> 1. Indentation uses SPACES, not tabs. Tabs will break YAML.
>
> 2. The number of spaces doesn't matter (2 or 4 are common), but it must be CONSISTENT within the same level.
>
> 3. Child items are indented MORE than their parent. If two lines are at the same indent, they are siblings.
>
> 4. One misaligned space can change the meaning of a document. Most YAML errors are from this.

The #1 YAML trap

This is valid YAML:

```
on:
  push:
    branches: [main]
```

This is ALSO valid YAML, but means something completely different:

```
on:
  push:
  branches: [main]     # <-- now 'branches' is a sibling of 'push', not its child
```

In the second version, 'push' has no value (it's implicitly null), and 'branches: [main]' is a sibling key of 'push' under 'on'. This is a syntactically valid YAML document but a semantically wrong workflow. GitHub Actions will not trigger on push — it will fail silently.

> ✅ **Tip**
> When you hit 'my workflow won't trigger' or 'GitHub says my file is invalid', paste your YAML into an online validator (like yamllint.com) or use the VS Code YAML extension. Both will highlight indentation issues that are invisible to the human eye.

6.2 — GitHub Actions Expression Syntax

Inside a workflow, you can reference values using ${{ ... }} syntax. These expressions are evaluated by GitHub before the job runs.

Common expression patterns

```
# Referencing a secret
aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}

# Referencing a built-in context
image-tag: ${{ github.sha }}          # commit SHA
branch: ${{ github.ref }}              # refs/heads/dev
actor: ${{ github.actor }}             # username who pushed

# Referencing another job's output
image: ${{ needs.build.outputs.image-tag }}

# Conditionals (if:)
if: github.ref == 'refs/heads/main'

# String interpolation
name: Deploy ${{ github.ref_name }}    # "Deploy main"
```

Common contexts you will use

Context	Contains
github.*	Info about the workflow run: branch, commit SHA, actor, event name, repo.
secrets.*	Values from your repo's Secrets configuration.
env.*	Environment variables set at the workflow, job, or step level.
steps.<id>.outputs.*	Values set by a previous step using echo "name=value" >> $GITHUB_OUTPUT.
needs.<job>.outputs.*	Values set by a previous job in that job's 'outputs:' block.
job.*	Info about the current job.
runner.*	Info about the runner: OS, temp directory, arch.

Step 19 — Create Your First Workflow

Create .github/workflows/ci.yml with this content. We are starting small — just a single job that checks out the code, installs dependencies, and runs the unit test. We will add more jobs in later parts.

```
name: CI Pipeline

on:
  push:
    branches: [main, dev]
  pull_request:
    branches: [main]

jobs:
  test:
    name: Lint and Test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'

      - name: Install dependencies
        working-directory: ./app
        run: npm ci

      - name: Run unit tests
        working-directory: ./app
        run: npm test
```

6.3 — Line-by-Line Walkthrough

name: CI Pipeline

Human-readable name. Shown in the Actions tab of your repo. Can be anything — this is just a label.

on:

Defines the triggers — what events cause this workflow to run. We specify two:

- `push: branches: [main, dev]` — run on pushes to main or dev. Pushes to other branches (like feature/xyz) are ignored.
- `pull_request: branches: [main]` — also run when a PR is opened or updated that targets main. This lets us test PR code BEFORE it is merged.
-

🔍 **Deep Dive**

Why both 'push' and 'pull_request'? They catch different workflows.

On a 'push' to dev, we want CI to run. But if someone opens a PR from dev to main, we ALSO want CI to run on the merged result — the combined code might behave differently than either branch alone. pull_request triggers on that merge preview.

Without pull_request, you would only test the branches in isolation. Bugs that appear only after merging would slip through.

On a push to main (after merge), CI runs again — this is normal and expected. It's cheap insurance against race conditions in the merge process.

jobs:

Lists the jobs. We have one: test. The string 'test' is the internal job ID — used to reference this job from other jobs (needs: test).

name: Lint and Test

A more descriptive name shown in the UI. Separate from the internal job ID.

runs-on: ubuntu-latest

The OS of the VM that runs this job. GitHub provides ubuntu-latest, windows-latest, and macos-latest for free (Ubuntu is by far the cheapest and fastest). The VM is fresh for every run — no state persists between runs.

steps:

The ordered list of steps in this job. Each step is either a 'uses:' (calls a reusable action) or a 'run:' (executes a shell command).

uses: actions/checkout@v4

Checks out your repository onto the runner. Without this step, the runner's disk is empty — it has no code to work with. checkout@v4 is the current major version. Pinning to a specific version is good practice so a breaking change in @main doesn't break your workflow.

uses: actions/setup-node@v4 with node-version: '18'

Installs Node.js 18 on the runner. The runner comes with many tools pre-installed, but specific versions may not be what you need. setup-node lets you choose. 'with:' passes parameters to the action — in this case, which Node version to install.

run: npm ci (inside app directory)

npm ci is a stricter, faster version of npm install designed for CI environments. Differences:

- npm ci requires package-lock.json to exist. npm install works without it.
- npm ci deletes node_modules first and does a clean install. npm install may leave stale packages.

- npm ci fails if package.json and package-lock.json disagree. npm install silently updates the lock file.
- npm ci is typically 2x faster because of the clean-install assumption.

working-directory: ./app tells GitHub to run this command inside ./app instead of the repo root. Same as typing 'cd app' first.

run: npm test

Runs the 'test' script from package.json, which is 'mocha server.test.js --exit'. If any test fails, mocha exits non-zero, the step fails, the job fails, and the workflow fails.

Step 20 — Commit, Push, and Watch It Run

```
git checkout dev                # make sure we're on dev
git add .github/workflows/ci.yml
git commit -m "Add initial CI workflow with unit tests"
git push origin dev
```

Open your repo on GitHub and click the Actions tab. You should see a workflow run appear called 'CI Pipeline' with your commit message. Click into it to watch it execute.

As the pipeline runs, each step shows one of these states:

- Yellow spinner = currently running
- Green checkmark = passed
- Red X = failed (click to see logs)
- Gray dash = skipped (previous step failed)

Click the 'Lint and Test' job to see each step's output. You should see npm installing dependencies, then mocha printing '1 passing'.

> 📝 **Note**
> The workflow file must exist on the branch you are pushing to. If you add ci.yml on main but only push to dev, the dev pipeline will not see the workflow until ci.yml is also on dev. We pushed to dev first, which is correct.

Part 7 — Integration Tests with Service Containers

Unit tests are great for pure logic but miss entire categories of bugs: wrong Redis command names, bad connection strings, serialization mismatches, timeouts. For these, you need to run your code against a real dependency.

7.1 — What Service Containers Do

GitHub Actions lets you declare 'service containers' — Docker containers that run alongside your job on the same runner. They start before any of your job's steps run, stop when the job ends, and are reachable from your job's steps via localhost.

In our case, we declare Redis as a service container. When the test job starts, GitHub first runs docker run -d -p 6379:6379 redis:7-alpine on the runner, waits for Redis to be healthy, then starts our test steps. Our integration tests connect to localhost:6379 and hit the real Redis server. When the job finishes, Docker stops and removes the container. Zero cleanup.

Step 21 — Replace the Test Job with an Expanded Version

Replace the test job in .github/workflows/ci.yml with this (keep the 'name:' and 'on:' blocks at the top — only replace the 'test:' job):

```
  test:
    name: Lint and Test
    runs-on: ubuntu-latest

    services:
      redis:
        image: redis:7-alpine
        ports:
          - 6379:6379
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'

      - name: Install dependencies
        working-directory: ./app
        run: npm ci

      - name: Run unit tests
```

```
        working-directory: ./app
        run: npm test

      - name: Run integration tests
        working-directory: ./app
        env:
          REDIS_HOST: localhost
        run: npm run test:integration
```

7.2 — Understanding Every Field

services: redis:

Declares a service container named 'redis'. The name becomes the DNS hostname from inside other containers — but since our test code runs directly on the runner (not in a container), we use localhost instead.

image: redis:7-alpine

The Docker image to run. Same version as what we use in production. Using the same version in test as production is important — Redis 6 and Redis 7 have subtle command-level differences.

ports: - 6379:6379

Port mapping. Host:Container. Maps the container's 6379 to the runner's localhost:6379. Without this line, Redis runs but we cannot reach it from our test code.

options: >-

Extra flags to pass to docker run. The >- is YAML's folded block scalar indicator — newlines in the value become spaces. So this:

```
options: >-
  --health-cmd "redis-cli ping"
  --health-interval 10s
  --health-timeout 5s
  --health-retries 5
```

...becomes equivalent to:

```
options: --health-cmd "redis-cli ping" --health-interval 10s --health-timeout 5s
--health-retries 5
```

The folded form is just easier to read.

Health check flags

Flag	Meaning
--health-cmd "redis-cli ping"	The command Docker runs inside the container to check health. 'redis-cli ping' returns PONG when Redis is ready to serve.

Flag	Meaning
--health-interval 10s	Run the health check every 10 seconds.
--health-timeout 5s	Each health check must complete within 5 seconds or it is considered failed.
--health-retries 5	Try up to 5 times before marking the container unhealthy. GitHub Actions will wait for the container to be healthy before starting any step.

env: REDIS_HOST: localhost

Sets an environment variable just for this step. Our integration test reads process.env.REDIS_HOST to know where to connect. Setting REDIS_HOST=localhost tells it to connect to localhost:6379, which is where the Redis container is listening.

npm run test:integration

Runs the integration test script (the 10-second-timeout one). npm run is how you invoke scripts that are not built-in (start, test, install). For custom script names like 'test:integration', you need 'npm run' — just 'npm test:integration' would not work.

Step 22 — Commit and Verify

```
git add .github/workflows/ci.yml
git commit -m "Add integration tests with Redis service container"
git push origin dev
```

In the Actions tab, watch the new run. The job detail page now shows TWO sections:

- **Set up job** — starts the Redis service container and waits for its health check.
- **Your steps** — checkout, setup-node, install, unit test, integration test.

Click 'Initialize containers' (or the services section) to see Docker pulling the Redis image and starting it. Then each test step runs. Integration tests should show '3 passing'.

> ✅ **Tip**
>
> If integration tests fail with ECONNREFUSED, the Redis service did not start in time. Check the 'Initialize containers' section of the log — you should see 'redis Container Started'. If you see an error, verify the image name and the ports mapping.
>
> If integration tests fail with 'Unknown command' errors, your code might be using a Redis command that doesn't exist in Redis 7. Check the Redis docs.

Part 8 — Build, Scan, Push to ECR

This is where security becomes real. We will build Docker images and push them to ECR, but not before scanning them for vulnerabilities. This part has more depth on the security aspects than any other.

8.1 — Why Scan Images?

A Docker image is not just your code. It contains:

- A Linux distribution (Alpine, in our case) with system libraries, compilers, package managers.
- A language runtime (Node.js 18) which has its own bundled dependencies.
- Your npm dependencies (express, ioredis), plus THEIR transitive dependencies.
- Your own code.

Every layer is a potential source of vulnerabilities. Maybe the version of OpenSSL bundled in your base image has a known CVE. Maybe a package 5 levels deep in your npm dependency tree has a remote code execution bug. You did not write these bugs, but you are shipping them.

What a CVE is

CVE = Common Vulnerabilities and Exposures. Every disclosed security vulnerability in open-source software is assigned a CVE number, like CVE-2024-12345. Public databases track which software versions are affected and which have fixes.

CVEs have severity ratings:

Severity	Typical Meaning
CRITICAL	Remote exploitation without authentication, root privileges. Fix immediately. Examples: unauthenticated RCE, heap corruption in TLS.
HIGH	Significant impact but requires specific conditions. Fix within days. Examples: authenticated RCE, privilege escalation.
MEDIUM	Limited impact or requires many preconditions. Fix within weeks. Examples: DoS, information disclosure.
LOW	Minor impact. Fix when convenient. Examples: minor info leaks.
UNKNOWN / NONE	Not assessed or no severity assigned. Rarely significant.

8.2 — What Trivy Does

Trivy is an open-source scanner by Aqua Security. Given a Docker image, it:

34. Extracts the image layers and identifies the base OS (Alpine, Debian, etc.).
35. Reads the OS package database (/lib/apk/db/installed for Alpine) to list installed packages and versions.
36. Reads language-specific lockfiles (package-lock.json, Gemfile.lock, go.sum) to list application dependencies.
37. Cross-references every package+version against the public CVE database.
38. Reports all matches, grouped by severity.

Trivy runs in seconds. It does not execute your code or the image — it is a purely static analysis, reading filesystem metadata.

Step 23 — Add the Build Job

Append this job to .github/workflows/ci.yml, at the same indentation level as the 'test:' job (inside the 'jobs:' block):

```
  build:
    name: Build, Scan, and Push to ECR
    needs: test
    runs-on: ubuntu-latest
    outputs:
      image-tag: ${{ steps.meta.outputs.sha }}

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ secrets.AWS_REGION }}

      - name: Login to Amazon ECR
        id: ecr-login
        uses: aws-actions/amazon-ecr-login@v2

      - name: Set image tag
        id: meta
        run: echo "sha=${GITHUB_SHA::8}" >> $GITHUB_OUTPUT

      - name: Build backend image
        run: |
          docker build \
            -t ${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.${{ secrets.AWS_REGION
}}.amazonaws.com/k8s-backend:${{ steps.meta.outputs.sha }} \
            ./app

      - name: Build frontend image
        run: |
```

```
          docker build \
            -t ${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.${{ secrets.AWS_REGION
}}.amazonaws.com/k8s-frontend:${{ steps.meta.outputs.sha }} \
            ./app/frontend

      - name: Scan backend image with Trivy
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: ${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.${{ secrets.AWS_REGION
}}.amazonaws.com/k8s-backend:${{ steps.meta.outputs.sha }}
          format: 'table'
          exit-code: '1'
          severity: 'CRITICAL,HIGH'
          ignore-unfixed: true

      - name: Scan frontend image with Trivy
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: ${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.${{ secrets.AWS_REGION
}}.amazonaws.com/k8s-frontend:${{ steps.meta.outputs.sha }}
          format: 'table'
          exit-code: '1'
          severity: 'CRITICAL,HIGH'
          ignore-unfixed: true

      - name: Push backend image
        run: |
          docker push ${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.${{ secrets.AWS_REGION
}}.amazonaws.com/k8s-backend:${{ steps.meta.outputs.sha }}

      - name: Push frontend image
        run: |
          docker push ${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.${{ secrets.AWS_REGION
}}.amazonaws.com/k8s-frontend:${{ steps.meta.outputs.sha }}
```

8.3 — Line-by-Line Explanation

needs: test

This job waits for the 'test' job to complete successfully. If test fails, build is skipped entirely. This is how we enforce 'tests first' — no image is ever built for broken code.

outputs: image-tag: ...

Declares that this job produces an output named 'image-tag' that other jobs can read. The value is whatever the step with id 'meta' outputs for 'sha'. Our deploy jobs will read this value to know which image tag to deploy.

aws-actions/configure-aws-credentials@v4

Official AWS action. It takes an access key and secret, and sets up the AWS CLI and Docker to use them. After this step runs, any aws or docker command in the job can talk to your AWS account.

aws-actions/amazon-ecr-login@v2

Logs Docker into your ECR. Runs aws ecr get-login-password internally and pipes it into docker login. After this step, docker push works without further authentication.

id: meta and $GITHUB_OUTPUT

```
- name: Set image tag
  id: meta
  run: echo "sha=${GITHUB_SHA::8}" >> $GITHUB_OUTPUT
```

This step produces an output. Breakdown:

- `id: meta` — gives the step an ID so other steps can reference its outputs.
- `${GITHUB_SHA::8}` — bash parameter expansion. GITHUB_SHA is the full 40-character commit hash; ::8 takes the first 8 characters. So 'a3f9c21b4d...' becomes 'a3f9c21b'.
- `>> $GITHUB_OUTPUT` — appends to a special file GitHub creates for outputs. GitHub reads this file after the step finishes and makes each 'key=value' line available as steps.meta.outputs.<key>.

Later steps read this via ${{ steps.meta.outputs.sha }}.

🔍 **Deep Dive**

Why tag images with commit SHAs instead of 'latest'?

1. Uniqueness. Every build produces a unique tag. You can never accidentally deploy the wrong version.

2. Traceability. Given a tag, you can git checkout that commit to see exactly what code is running in production.

3. Rollback. If v3 is broken, you can redeploy v2 or any earlier tag immediately.

4. Kubernetes behavior. K8s caches images by tag. If you push a new image with the same tag, K8s may not pull it ('it's already the same tag, so it must be the same image'). Unique tags force fresh pulls every time.

5. Audit. A malicious insider cannot secretly push a different image over your 'latest' tag — they would have to push with a different commit SHA, which is visible in git history.

This is why 'never use :latest in CI/CD' is a universally recommended practice.

Trivy steps

We run Trivy twice — once per image. Each run passes the same parameters:

Parameter	What It Does
image-ref	The image to scan. Here we use the same ECR URI we tagged — Trivy scans the image locally before it is pushed.

Parameter	What It Does
format: 'table'	Output format. 'table' produces a human-readable table in the logs. Other options: 'json' (machine-readable), 'sarif' (uploads to GitHub Security tab).
exit-code: '1'	CRITICAL FLAG. If Trivy finds vulnerabilities matching the severity filter, exit 1 (failure). This makes the step fail, which makes the job fail, which stops the pipeline. Without this flag, Trivy would report vulns but not block anything.
severity: 'CRITICAL,HIGH'	Only fail on CRITICAL and HIGH severity. MEDIUM and LOW are still logged but don't block the pipeline.
ignore-unfixed: true	Ignore vulnerabilities that have no fix available. Otherwise the pipeline blocks on issues you literally cannot resolve (e.g., a CVE in Alpine that upstream hasn't patched).

Example Trivy output in logs

```
k8s-backend:a3f9c21b (alpine 3.18.0)

Total: 3 (HIGH: 2, CRITICAL: 1)

┌─────────────────┬────────────────────┬──────────┬─────────────────┬──────────────────────┐
│ Library         │ Vulnerability      │ Severity │ Installed       │ Fixed In             │
├─────────────────┼────────────────────┼──────────┼─────────────────┼──────────────────────┤
│ openssl         │ CVE-2024-XXXX      │ CRITICAL │ 3.1.4           │ 3.1.5                │
│ curl            │ CVE-2024-YYYY      │ HIGH     │ 8.4.0           │ 8.5.0                │
│ libcrypto3      │ CVE-2024-ZZZZ      │ HIGH     │ 3.1.4           │ 3.1.5                │
└─────────────────┴────────────────────┴──────────┴─────────────────┴──────────────────────┘

ERROR: Trivy found vulnerabilities. Failing step.
```

How to fix when Trivy blocks you

If the pipeline fails with CVE errors, you have three options:

39. **Update the base image.** Change FROM node:18-alpine to FROM node:18-alpine3.19 or whatever the newest Alpine patch is. Rebuild.
40. **Update your dependencies.** Run npm audit fix inside the app directory, commit the updated package-lock.json, and push.
41. **Add to an ignore list.** If the CVE does not actually affect your app (e.g., you don't use the vulnerable feature), you can add the CVE ID to a .trivyignore file. Use sparingly and document why.

Push steps (only run if scan passed)

The push steps are straight docker push commands. Because the scan steps come before push and fail the job if vulnerabilities are found, the push steps only execute if the image passed the scan. The exact flow:

- Build backend image → SUCCESS
- Build frontend image → SUCCESS
- Scan backend → if CRITICAL/HIGH found, FAIL (stop here)
- Scan frontend → if CRITICAL/HIGH found, FAIL (stop here)
- Push backend → only runs if both scans passed
- Push frontend → only runs if both scans passed

> ⚠️ **Warning**
> The order of steps matters. If you put 'push' before 'scan', a vulnerable image reaches ECR before Trivy ever runs. The scan would still fail the pipeline, but the image is already in your registry where someone might pull it.
>
> Always: build → scan → push. Never: build → push → scan.

Step 24 — Commit, Push, Verify

```
git add .github/workflows/ci.yml
git commit -m "Add build, scan, and push job"
git push origin dev
```

The Actions tab now shows two jobs connected by an arrow: test → build. Click the build job to see Trivy's output in the scan steps. The final output should show images pushed with your commit SHA as the tag.

Verify the images are in ECR:

```
aws ecr list-images --repository-name k8s-backend  --region us-east-1
aws ecr list-images --repository-name k8s-frontend --region us-east-1
```

You should see v1 (from Step 12) plus the new commit-SHA tag.

Part 9 — Deploy to Dev and Prod

Two more jobs, one for each environment. They look almost identical — only three lines differ.

Step 25 — Add the deploy-dev Job

Append to .github/workflows/ci.yml:

```
  deploy-dev:
    name: Deploy to Dev
    needs: build
    if: github.ref == 'refs/heads/dev'
    runs-on: ubuntu-latest
    environment: dev

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ secrets.AWS_REGION }}

      - name: Update kubeconfig
        run: |
          aws eks update-kubeconfig \
            --name ${{ secrets.EKS_CLUSTER_NAME }} \
            --region ${{ secrets.AWS_REGION }}

      - name: Update deployments to new image
        run: |
          TAG=${{ needs.build.outputs.image-tag }}
          REG=${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.${{ secrets.AWS_REGION
}}.amazonaws.com

          kubectl -n dev set image deployment/backend  backend=$REG/k8s-backend:$TAG
          kubectl -n dev set image deployment/frontend frontend=$REG/k8s-frontend:$TAG

      - name: Wait for rollout
        run: |
          kubectl -n dev rollout status deployment/backend  --timeout=180s
          kubectl -n dev rollout status deployment/frontend --timeout=180s

      - name: Verify pods
        run: kubectl get pods -n dev -o wide
```

9.1 — Key Lines Explained

needs: build

Full dependency chain: deploy-dev needs build, which needs test. So: test → build → deploy-dev. If ANY upstream job fails, deploy-dev is skipped.

if: github.ref == 'refs/heads/dev'

Critical condition. This job only runs if the branch being pushed to is 'dev'. github.ref is the full reference name — for a branch push, it's 'refs/heads/<branch>'. So 'refs/heads/dev' means 'the dev branch.' Without this condition, a push to main would trigger a dev deployment, which is wrong.

> 🔍 **Deep Dive**
>
> github.ref vs github.ref_name — often confused.
>
> github.ref is the full reference: 'refs/heads/dev' for a branch push, 'refs/tags/v1.0' for a tag push, 'refs/pull/42/merge' for a PR.
>
> github.ref_name is just the name: 'dev', 'v1.0', '42/merge'.
>
> For branch filtering in if:, always compare against 'refs/heads/<branch>' using github.ref. It's explicit and unambiguous. Using github.ref_name = 'dev' works too but is slightly less robust (a tag named 'dev' would match).

environment: dev

Associates the job with the 'dev' GitHub Environment. Since we left dev with no protection rules, this has no gating effect — the job runs immediately. But it does record a deployment entry in the Environments tab, which is useful for audit and history.

aws eks update-kubeconfig

Writes a kubeconfig file to ~/.kube/config that teaches kubectl how to talk to your cluster. Every subsequent kubectl command in this job uses this config. The runner is fresh for every job, so kubeconfig from previous runs is gone — we must re-run this in every job that uses kubectl.

Env var capture in a run block

```
TAG=${{ needs.build.outputs.image-tag }}
REG=${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.${{ secrets.AWS_REGION }}.amazonaws.com
```

These two lines capture long values into short shell variables just for readability. The alternative is inline substitution in every kubectl command, which becomes very long. Inside 'run: |' blocks, you write real bash — you can use variables, conditionals, loops, anything bash supports.

kubectl set image

Updates the container image on an existing deployment. Does not touch replicas, env vars, probes, or any other field. Triggers a rolling update: Kubernetes creates new pods with the new image, waits for them to become ready, then terminates the old pods. Zero downtime.

Format: 'kubectl set image deployment/<name> <container-name>=<new-image>'. The container-name must match the 'name:' field inside the deployment's container spec. In our manifests, the container is named 'backend' for the backend and 'frontend' for the frontend.

kubectl rollout status

Blocks the step until the rolling update completes, or until the timeout expires. Completes means all new pods are ready. If pods fail to start (e.g., ImagePullBackOff, or the new code crashes on startup), the step fails after the timeout, making the whole job fail. You know immediately that the deployment is broken.

Step 26 — Add the deploy-prod Job

Append below deploy-dev. The only differences are the if: condition, the environment:, and the namespace in kubectl commands:

```
  deploy-prod:
    name: Deploy to Production
    needs: build
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: prod

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ secrets.AWS_REGION }}

      - name: Update kubeconfig
        run: |
          aws eks update-kubeconfig \
            --name ${{ secrets.EKS_CLUSTER_NAME }} \
            --region ${{ secrets.AWS_REGION }}

      - name: Update deployments to new image
        run: |
          TAG=${{ needs.build.outputs.image-tag }}
          REG=${{ secrets.AWS_ACCOUNT_ID }}.dkr.ecr.${{ secrets.AWS_REGION
}}.amazonaws.com

          kubectl -n prod set image deployment/backend  backend=$REG/k8s-backend:$TAG
          kubectl -n prod set image deployment/frontend frontend=$REG/k8s-frontend:$TAG

      - name: Wait for rollout
        run: |
          kubectl -n prod rollout status deployment/backend  --timeout=180s
          kubectl -n prod rollout status deployment/frontend --timeout=180s

      - name: Verify pods
        run: kubectl get pods -n prod -o wide
```

The three differences from deploy-dev:

- `if: github.ref == 'refs/heads/main'` — only runs on main branch pushes.

- `environment: prod` — attaches to the protected environment, triggering the approval gate.
- Every `-n dev` becomes `-n prod`.

Step 27 — Test the Full Dev Pipeline

Make a visible change so you can prove the deploy actually worked. Edit app/frontend/index.html:

```
<h1>CI/CD Lab — [YOUR NAME] — DEV</h1>
```

Commit and push:

```
git add .
git commit -m "Add DEV label to frontend for pipeline test"
git push origin dev
```

In the Actions tab, watch three jobs run: test → build → deploy-dev. Each has its own box in the workflow visualizer. deploy-prod does NOT appear because we pushed to dev, not main.

After deploy-dev finishes, verify the new code is running:

```
kubectl get pods -n dev
# Pods should have been recreated with the new image. Check the AGE column.

POD=$(kubectl -n dev get pod -l app=frontend -o jsonpath='{.items[0].metadata.name}')
kubectl -n dev exec $POD -- curl -s localhost:80 | grep "DEV"
# Should print the <h1> line containing "DEV"
```

Port-forward to the frontend for a browser preview:

```
kubectl port-forward -n dev svc/frontend-service 8080:80
# Open http://localhost:8080 in your browser
# You should see "CI/CD Lab — Your Name — DEV"
# Ctrl+C to stop the port-forward
```

Step 28 — Test the Full Prod Pipeline

Merge dev into main to trigger the prod pipeline:

```
git checkout main
git merge dev
git push origin main
```

Watch the Actions tab. The pipeline runs test → build, then deploy-prod appears with a yellow 'Waiting' status. Click into deploy-prod:

- You'll see a message: 'Deployment protection rules: Required reviewers (you)'
- A green 'Review deployments' button.

Click Review deployments → select prod → optionally leave a comment → click 'Approve and deploy'.

The deploy runs and completes. Verify prod:

```
kubectl get pods -n prod
POD=$(kubectl -n prod get pod -l app=frontend -o jsonpath='{.items[0].metadata.name}')
kubectl -n prod exec $POD -- curl -s localhost:80 | grep -i "CI/CD Lab"
```

> 📝 **Note**
> You may notice that prod shows 'DEV' in the title too — that's expected. When we merged dev into main, we also merged our 'DEV' label change. In a real project, you'd typically use a feature-flag or environment variable to differentiate environments, rather than baking 'DEV' into the HTML. This is a shortcut for the lab.

Part 10 — Rolling Back a Bad Deployment

You deploy. Two minutes later, users report errors. The new code has a bug. Restoring service fast matters more than finding the root cause — fix first, investigate second. Kubernetes makes this trivial.

10.1 — How Kubernetes Rollback Works

Every time a Deployment's pod spec changes (new image, new env var, new config), Kubernetes records the previous spec as a revision. By default, Kubernetes keeps the last 10 revisions. 'Rollback' means 'apply a previous revision' — Kubernetes runs a new rolling update using the old spec.

Rollback takes 10–30 seconds because it is just restarting pods with a different image. No rebuild, no re-scan, no ECR pull (image is already cached on the node).

Step 29 — View Rollout History

```
kubectl rollout history deployment/backend -n prod

# Example output:
# REVISION  CHANGE-CAUSE
# 1         <none>
# 2         <none>
# 3         <none>
```

Three revisions exist — probably: (1) initial deploy from Step 13, (2) first pipeline deploy from Step 28, (3) any subsequent deploys. To see what each revision contains:

```
kubectl rollout history deployment/backend -n prod --revision=2
# Shows the full pod spec for revision 2 — image, env, probes, resources.
```

Step 30 — Roll Back

```
# Roll back to the revision just before the current one
kubectl rollout undo deployment/backend -n prod

# Or target a specific revision
kubectl rollout undo deployment/backend -n prod --to-revision=2

# Watch it happen
kubectl rollout status deployment/backend -n prod
kubectl get pods -n prod
# You should see old pods terminating, new pods (with the old image) starting up.
```

Done. Production is now running the previous version. This takes under 30 seconds. Compare to waiting for a full pipeline: test (1 min) + build (2 min) + scan (1 min) + push (1 min) + approve + deploy (2 min) = 7+ minutes if you had to fix the code first.

Step 31 — Also Revert in Git

Critical: Kubernetes rollback fixes the running cluster, but the bad commit is still in your main branch. If someone triggers another deploy, the pipeline will rebuild and redeploy the bad code.

After any Kubernetes rollback, also revert the bad commit in Git:

```
git log --oneline -5
# Find the bad commit SHA

git revert <bad-commit-sha>
# Git opens your editor for a commit message. Save and close.

git push origin main
# This triggers the pipeline — which will deploy the REVERTED code
# (same as what you rolled back to in Kubernetes).
# The deploy-prod gate will pause for your approval again.
```

> ✅ **Tip**
>
> git revert creates a NEW commit that undoes the bad one. History is preserved — you can still see the bad commit and the revert in git log. This is safe for shared branches.
>
> git reset rewrites history by deleting commits. Never use reset on a shared branch — it can destroy teammates' work. Only use reset on your own local branches before pushing.

Part 11 — Terraform in the Pipeline

Application code is automated. But what about infrastructure? If someone modifies an S3 bucket, adds a security group rule, or changes an EC2 instance type, that change should also go through code review and pipeline automation. Humans running terraform apply from laptops is how production goes down at 2 AM.

11.1 — The Terraform Pipeline Pattern

```
PR opened or updated              PR merged to main
        |                                 |
        v                                 v
  Trigger: pull_request            Trigger: push (main)
        |                                 |
        v                                 v
  terraform init                    terraform init
  terraform fmt -check              terraform apply -auto-approve
  terraform plan                       (gated by prod approval)
        |                                 |
        v                                 v
  Post plan output as               Infrastructure changes
  a comment on the PR               are now live in AWS
```

On a PR, show what WILL change. Merging triggers the actual change, gated by approval.

Step 32 — Create a Minimal terraform/ Starter

We provision a single S3 bucket as a demo. The pipeline pattern is the point — the specific resource doesn't matter. You can later replace this with whatever real infrastructure you need to manage.

terraform/versions.tf

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  # Remote state — uncomment AFTER creating the S3 bucket and
  # DynamoDB table in Step 33. Commented initially to avoid the
  # chicken-and-egg problem of Terraform managing its own backend.
  #
  # backend "s3" {
  #   bucket         = "cicd-lab-tfstate-<your-unique-suffix>"
  #   key            = "prod/terraform.tfstate"
  #   region         = "us-east-1"
  #   dynamodb_table = "cicd-lab-tflocks"
```

```
  #   encrypt        = true
  # }
}

provider "aws" {
  region = var.region
}
```

terraform/variables.tf

```
variable "region" {
  type        = string
  default     = "us-east-1"
  description = "AWS region for all resources."
}

variable "bucket_suffix" {
  type        = string
  description = "Globally-unique suffix for the demo S3 bucket. S3 bucket names must be
globally unique across all AWS accounts — use your initials + random chars."
}

variable "environment" {
  type        = string
  default     = "lab"
  description = "Environment tag applied to resources."
}
```

terraform/main.tf

```
resource "aws_s3_bucket" "demo" {
  bucket = "cicd-lab-demo-${var.bucket_suffix}"

  tags = {
    Environment = var.environment
    ManagedBy   = "terraform"
    Lab         = "010"
  }
}

resource "aws_s3_bucket_versioning" "demo" {
  bucket = aws_s3_bucket.demo.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_public_access_block" "demo" {
  bucket = aws_s3_bucket.demo.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
```

```
}

output "bucket_name" {
  value = aws_s3_bucket.demo.id
}
```

terraform/terraform.tfvars

Your values file. The bucket_suffix needs to be globally unique:

```
bucket_suffix = "aub-abc123"    # CHANGE THIS — include your initials and some random
characters
environment   = "lab"
```

Step 33 — Create the Remote State Backend (Recommended)

When Terraform runs in a pipeline, state cannot live on a laptop — the runner is destroyed after each job. State must be in S3 (shared, durable) with a DynamoDB table for locking (prevents two parallel applies corrupting state).

Create both by hand — once — before enabling the backend:

```
# Pick a globally-unique name (S3 bucket names are shared worldwide)
export TF_STATE_BUCKET=cicd-lab-tfstate-aub-abc123

aws s3api create-bucket \
  --bucket $TF_STATE_BUCKET \
  --region us-east-1

aws s3api put-bucket-versioning \
  --bucket $TF_STATE_BUCKET \
  --versioning-configuration Status=Enabled

aws s3api put-public-access-block \
  --bucket $TF_STATE_BUCKET \
  --public-access-block-configuration \

"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=t
rue"

aws dynamodb create-table \
  --table-name cicd-lab-tflocks \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --region us-east-1
```

Then uncomment the backend "s3" block in versions.tf, fill in your real bucket name, and run terraform init locally once. Terraform will ask to migrate existing state (if any) into S3 — answer yes.

🔍 **Deep Dive**

Why DynamoDB for state locking? Terraform needs a strongly consistent lock. S3 used to not provide this, so a separate DynamoDB table was used for locks while state itself lived in S3.

As of 2024, S3 now supports strong consistency and native locking, and Terraform 1.10+ can use S3 alone. But most existing setups and documentation still show DynamoDB, so we keep it for compatibility.

Either way: the goal is 'only one terraform apply runs at a time, to avoid state corruption.' Locking enforces that.

Step 34 — Create the Terraform Workflow

Create .github/workflows/terraform.yml. This workflow triggers only on changes to the terraform/ directory, avoiding a Terraform run for every application code change:

```
name: Terraform

on:
  pull_request:
    branches: [main]
    paths:
      - 'terraform/**'
  push:
    branches: [main]
    paths:
      - 'terraform/**'

env:
  TF_WORKING_DIR: ./terraform

jobs:
  plan:
    name: Terraform Plan
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v4

      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ secrets.AWS_REGION }}

      - uses: hashicorp/setup-terraform@v3

      - name: Terraform fmt
```

```
        working-directory: ${{ env.TF_WORKING_DIR }}
        run: terraform fmt -check -recursive

      - name: Terraform init
        working-directory: ${{ env.TF_WORKING_DIR }}
        run: terraform init

      - name: Terraform plan
        id: plan
        working-directory: ${{ env.TF_WORKING_DIR }}
        run: terraform plan -no-color -out=tfplan

      - name: Post plan to PR
        uses: actions/github-script@v7
        with:
          script: |
            const output = `#### Terraform Plan 📋
            \`\`\`
            ${{ steps.plan.outputs.stdout }}
            \`\`\`
            *Triggered by @${{ github.actor }}*`;
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: output
            });

  apply:
    name: Terraform Apply
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: prod
    steps:
      - uses: actions/checkout@v4

      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ secrets.AWS_REGION }}

      - uses: hashicorp/setup-terraform@v3

      - name: Terraform init
        working-directory: ${{ env.TF_WORKING_DIR }}
        run: terraform init

      - name: Terraform apply
        working-directory: ${{ env.TF_WORKING_DIR }}
        run: terraform apply -auto-approve
```

11.2 — Key Lines Explained

paths: - 'terraform/**'

Path filter. This workflow only triggers when files matching terraform/** change. Without this, every application code change would run terraform plan — pointless and wasteful.

permissions: pull-requests: write

The plan job needs to post a comment on the PR. GitHub restricts what the GITHUB_TOKEN can do by default (read-only for most things). We explicitly grant write permission on pull-requests so the 'Post plan to PR' step can create a comment.

terraform fmt -check -recursive

Enforces consistent formatting. fmt with no flags would reformat files. -check reports whether any files need reformatting and exits non-zero if so. -recursive descends into subdirectories. Run terraform fmt locally before pushing to avoid this failing in CI.

terraform plan -no-color -out=tfplan

Computes what would change without changing anything. -no-color strips terminal escape codes (they render as garbage in PR comments). -out=tfplan saves the plan to a file — useful if you wanted to apply exactly this plan later, though we don't use it here.

actions/github-script@v7

Lets you run JavaScript that interacts with the GitHub API. We use it to POST a comment to the PR with the plan output. The special github.* API reference gives access to the full GitHub REST API.

environment: prod on apply

Same trick we used for application deploys: the prod environment has required reviewers, so terraform apply pauses for human approval before running. Critical — a Terraform typo can delete an entire VPC.

Step 35 — Test the Terraform Pipeline

Push the new terraform files to a feature branch and open a PR:

```
git checkout -b terraform-demo
git add terraform/ .github/workflows/terraform.yml
git commit -m "Add Terraform demo config and pipeline"
git push -u origin terraform-demo
```

On GitHub, open a PR from terraform-demo to main. The plan workflow runs. Within a minute, a comment appears on the PR showing the plan output — the S3 bucket that will be created.

Merge the PR. The apply workflow triggers, pauses for your prod approval, and then creates the S3 bucket. Verify:

```
aws s3api list-buckets --query 'Buckets[?contains(Name, `cicd-lab-demo`)]'
```

Part 12 — Branch Protection

The final lock. Prevent direct pushes to main so every change must go through a PR with passing CI.

Step 36 — Enable Branch Protection

In your repo → Settings → Branches → Add branch protection rule. Configure:

- **Branch name pattern:** `main`
- **Require a pull request before merging** — no direct pushes to main.
- **Require status checks to pass before merging** — select the 'Lint and Test' check.
- **Require branches to be up to date before merging** — ensures the PR is tested against current main, not stale main.
- **Do not allow bypassing the above settings** — applies rules even to admins.

After you save, anyone running git push origin main directly will be rejected. The only way to change main is:

42. Branch off dev.
43. Make changes, commit, push.
44. Open a PR to main.
45. Wait for CI to pass.
46. Get a review (optional but recommended).
47. Merge. This triggers the prod pipeline, which still needs your approval before deploying.

12.1 — The Complete Developer Workflow

With everything in place, here's the full flow a developer experiences:

48. Create a feature branch off dev: git checkout -b fix/some-bug dev.
49. Write code, commit: git commit -m 'Fix some bug'.
50. Push: git push -u origin fix/some-bug.
51. On GitHub, open a PR: fix/some-bug → dev.
52. CI runs (test + build + scan + push). Merge is blocked until it passes.
53. PR is approved, merged to dev. CI runs on dev. deploy-dev deploys automatically.
54. Developer verifies in dev namespace.
55. When ready for prod: open a PR dev → main.
56. CI runs on the PR. Merge is blocked until it passes.
57. PR is approved, merged to main. CI runs on main. deploy-prod pauses for approval.
58. Reviewer approves the prod deploy via the Environments UI.
59. Change goes live in prod.

Part 13 — Submission

Take the following screenshots in order. Your name must be visible on every browser screenshot. Combine them into a single PDF.

#	Screenshot	What It Should Show
1	Actions tab — dev run	Successful dev pipeline: test → build → deploy-dev, all green
2	Actions tab — prod run	Prod pipeline with the approval step visible
3	Approval dialog	The 'Review deployments' dialog for prod
4	Terminal — kubectl	kubectl get pods -n dev with recent AGE
5	Terminal — kubectl	kubectl get pods -n prod with recent AGE
6	Browser — dev	App running, title shows your name and 'DEV'
7	Browser — prod	App running in prod
8	Rollout history	kubectl rollout history deployment/backend -n prod (≥ 2 revisions)
9	Trivy output	Build job logs showing the Trivy scan table
10	Branch protection	Settings page showing main's protection rules
11	Terraform PR comment	A PR with terraform plan output as a comment
12	ci.yml	Full contents of .github/workflows/ci.yml in your repo

Submit: GitHub repository link + single PDF with all 12 screenshots via Moodle.

Part 14 — Clean Up

> ⚠️ **Warning**
> EKS control plane: $0.10/hour. Two t3.medium nodes: $0.08/hour. Total: about $4.30/day. A cluster forgotten for a week is $30. For a month: $130.
>
> Delete everything right after you finish the lab. You can recreate it in 20 minutes if you need to revisit anything.

Step 37 — Delete Kubernetes Resources

```
kubectl delete namespace dev
kubectl delete namespace prod
```

Step 38 — Destroy Terraform Resources

If you created the Terraform demo, destroy those resources before deleting the cluster:

```
cd terraform
terraform destroy -auto-approve
```

Step 39 — Delete the EKS Cluster

```
eksctl delete cluster --name cicd-lab --region us-east-1
# Takes about 10 minutes
```

Step 40 — Delete ECR Repositories

```
aws ecr delete-repository --repository-name k8s-frontend --force --region us-east-1
aws ecr delete-repository --repository-name k8s-backend  --force --region us-east-1
```

Step 41 — Delete the Terraform State Backend (If Created)

```
aws s3 rm s3://$TF_STATE_BUCKET --recursive
aws s3api delete-bucket --bucket $TF_STATE_BUCKET
aws dynamodb delete-table --table-name cicd-lab-tflocks --region us-east-1
```

Step 42 — Delete the IAM User

The AdministratorAccess IAM user is dangerous. Remove it:

```
# Delete access keys first
aws iam list-access-keys --user-name cicd-lab-github
aws iam delete-access-key --user-name cicd-lab-github --access-key-id <KEY_ID>

# Detach policies
aws iam detach-user-policy --user-name cicd-lab-github \
```

```
  --policy-arn arn:aws:iam::aws:policy/AdministratorAccess

# Delete the user
aws iam delete-user --user-name cicd-lab-github
```

Verify in the AWS Console: no EKS clusters, no EC2 instances, no load balancers, no ECR repos, no S3 buckets from this lab, no IAM user.

THE END